

1. Rational Agent: To maximize the expected value of its performance measure. (↗ guarantee best performance)

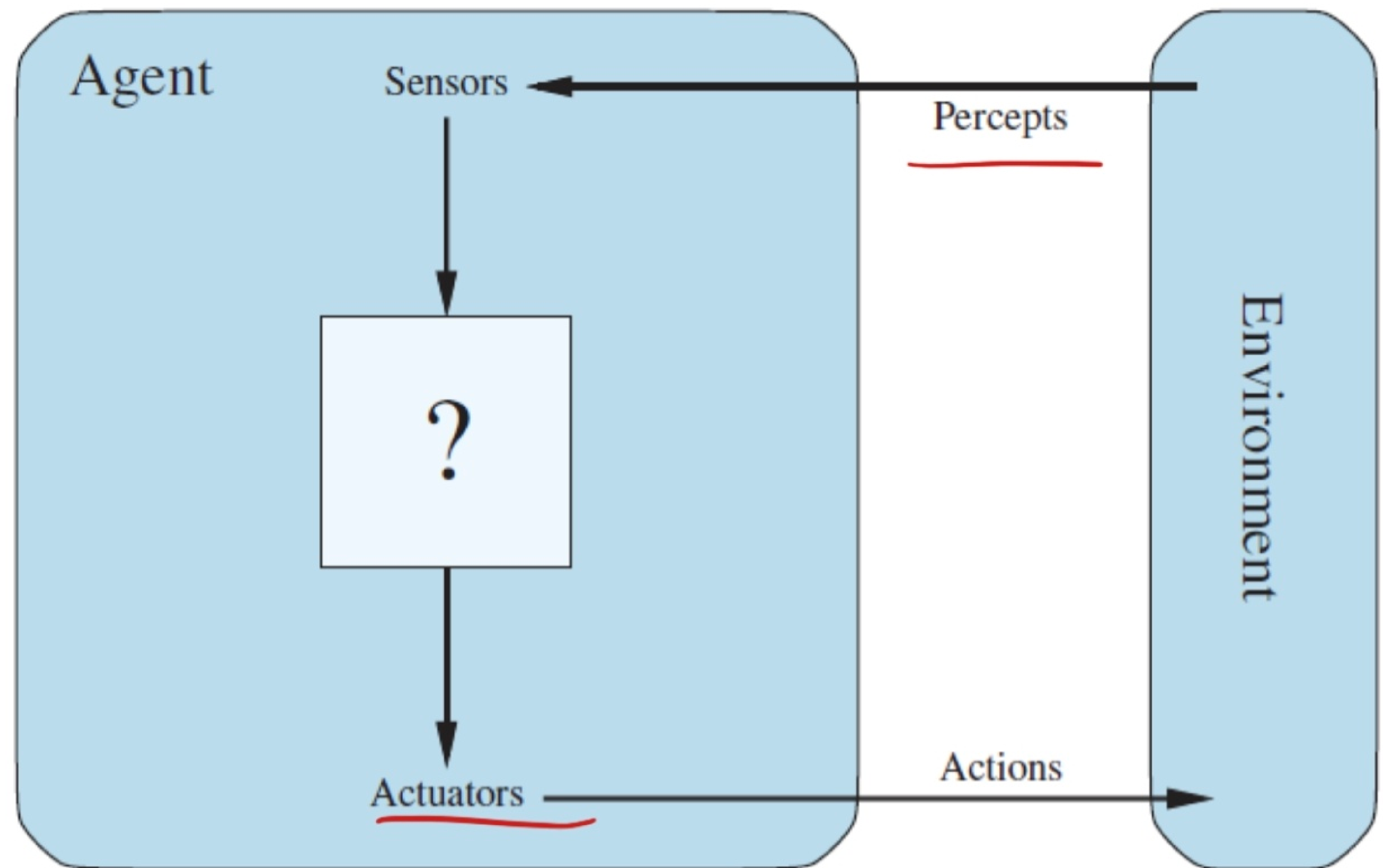
2. { Reflex Agent: No goals. Directly map states to actions.
 [Problem Solving Agent: Have goals. A sequence of actions.

Definition of Agent

An **agent** is anything that **perceives** its environment through **sensors** and can **act** on its environment through **actuators**

A **percept** is the agent's perceptual inputs at any given instance

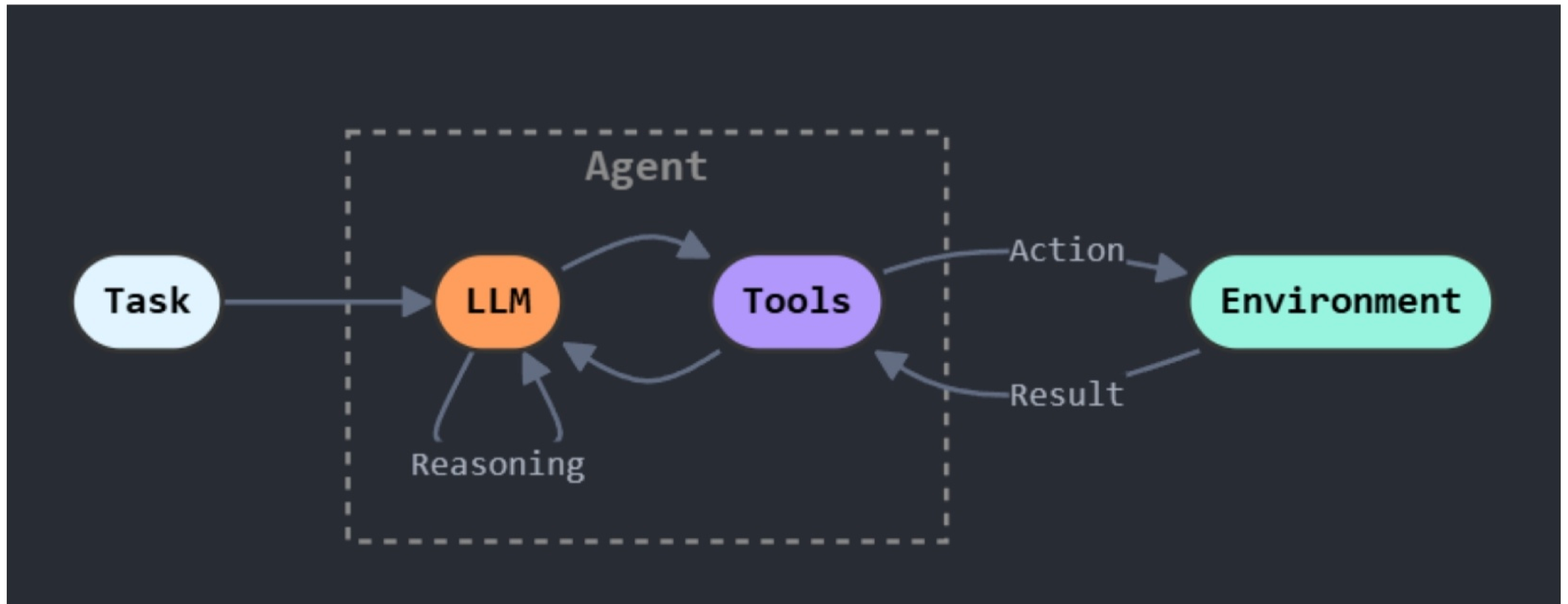
An **actuator** is the part of an agent that allows it to take action in the environment based on its decisions.



ReAct AI Agent



A **ReAct agent** (short for *Reasoning and Acting*) is a framework that enables LLMs to alternately generate explicit reasoning steps (thoughts) and perform actions (like API or tool calls) in a loop—synergizing internal reasoning with external execution to solve complex, multi-step tasks



1. { Tree search : Explore state space as a tree
Graph search : Track visited nodes to avoid revisit

2. Frontier: Queue of nodes to be expanded

3. Evaluations of search algorithm:

① Completeness: Does the algorithm always find a solution when it exists?

② Optimality: Does it find the lowest-cost (best) solutions among all possible ones?

③ Time complexity ④ Space complexity

4. Breadth-First Search (BFS) : Explore nodes in layers.

- Frontier as a FIFO queue.

5. Depth-First Search (DFS) : Expand the deepest node first.

- LIFO (Last in First out)

General search problems formulation

In general, a search problem is defined by:

1. **State space:** a set S containing all possible states of the environment.
2. **Initial state:** $s_i \in S$, indicating where the agent starts.
3. **Actions:** a set A containing possible moves from a state.
 $\forall s \in S: A = \text{ACTIONS}(s) = \text{actions executable in } s$
4. **Transition model:** Result of applying an action.
 $\forall a \in \text{ACTIONS}(s), \text{RESULT}(s, a) \rightarrow s', s' \text{ is called a successor of } s$
At state s , how many actions can be taken
5. **Action cost:** Cost of applying an action, $\text{ACTION-COST}(s, a, s')$, aligned with the performance measure
6. **Goal state(s):** Desired state(s) with a desired property
The agent uses a **goal test** to check if the current state is a goal state
 $\text{IS-GOAL}(s)$: s is a goal state if $\text{IS-GOAL}(s)$ is true

Summary: Tree search vs graph search



Aspect	Tree search	Graph search
Explored set	No (no memory of visited states)	Yes (maintains a set of visited states)
Memory use	Low	High (stores all visited states)
Time efficiency	Low (may revisit states many times)	High (avoids revisiting)
Risk of infinite loops	High (esp. in graphs with cycles)	Low (prevents loops via state tracking)
Implementation complexity	Simpler	More complex
When to use	- Memory-constraint systems - Cycles are impossible or negligible.	- Cyclic or redundant state spaces - Sufficient memory is available
Typical use cases	Scenarios with huge state spaces and no cycles, memory-limited systems	Most real-world problems with state reuse (e.g., pathfinding, games and puzzles)

Recap: Graph search vs Tree search

Tree search

```
function TREE-SEARCH(problem, strategy) return a solution, or failure
  Initialize frontier to the initial state of the problem
  loop do
    if the frontier is empty then return failure
    choose a node from the frontier for expansion according to strategy,
    and remove it from frontier
    if the node contains goal state then return the corresponding solution
    else expand the node, and add the resulting child nodes to the frontier
  end
```

Graph search

```
function GRAPH-SEARCH(problem, strategy) return a solution, or failure
  Initialize frontier to the initial state of the problem
  Initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a node from the frontier for expansion according to strategy,
    and remove it from frontier
    if the node contains goal state then return the corresponding solution
    add the node to the explored set
    expand the node, and add the resulting nodes to the frontier
    only if not in the frontier or explored set
  end
```


1. BFS is only effective in graphs where all edges have the same unit length.

Uniform-cost search (UCS, or Dijkstra's algorithm) can handle this by handling graphs with any non-negative edge length.

2. Dijkstra's algorithm has a greedy nature.

① It doesn't back track ② Fails in negative edge

3. Greedy-Best First Search: Not optimal or complete
Only rely on the heuristic function

BFS vs DFS vs Dijkstra's algorithm

Feature	BFS (Breadth-First Search)	DFS (Depth-First Search)	Dijkstra's (Uniform-cost search)
Search style	Explores all nearby nodes first (level by level)	Goes as deep as possible before backtracking	Expands the node with the lowest total cost from the source (initial state)
Data structure used	Queue (FIFO - First in, first out)	Stack (LIFO - Last in, first out)	Priority queue (based on path cost)
Will it always find a solution if one exists? (Completeness)	Yes, if a solution exists	Not guaranteed – may get stuck in deep or infinite paths	Yes
Will it find the shortest solution? (Optimality)	Yes, if all steps cost the same	No, may find a longer or less efficient path	Yes, even when edge costs differ
Can it handle different edge costs?	No – assumes all edges have equal cost	No – ignores cost	Yes – works with <u>any non-negative edge cost</u>
Time efficiency	Slower if the solution is deep	Can be faster if the solution is deep	Slower in large graphs with varying costs
Memory usage	High – remembers all paths at one level	Low – tracks only one path at a time	High – must track cumulative costs for all paths
Best for	Finding shortest paths (e.g., in maps) Small search space	Exploring puzzles or scenarios with deep solutions Large or deep search spaces	Finding the cheapest/least-cost path in weighted graphs (e.g., shortest time, lowest expense)
When to use	Need shortest route Equal step cost Enough memory	Memory is limited When solution may be far/deep Don't need shortest path	Need shortest route with different edge costs; can afford higher memory usage

Dijkstra's algorithm (uniform-cost search): Pseudocode

- Start with a **frontier (priority queue)** containing the initial state (source), with path cost = 0.
- Start with an **empty explored set**.
- **Repeat**:
 - If the frontier is empty, then no solution.
 - **Remove the frontier node** with the **lowest path cost from the source**.
 - If this node is the goal, return the solution.
 - **Add the node to the explored set**.
 - Expand the node:
 - For each child, compute its **total path cost from the source**.
 - If the child is not in the frontier or explored set, **add it to the frontier** with its cost.
 - If the child is already in the frontier with a **higher cost**, update it with the **lower cost**.

A node is marked as visited once it is removed from the frontier (if not goal)

Recap:

Search algorithms

Uniformed search

Breadth
first
search

Depth
first
search

Uniform
cost
search
(Dijkstra's
algorithm)

Informed search

Greedy
best-first
search

A* search

1. A^* search: Combine the actual cost to reach the node ($g(n)$) with the estimated cost to the goal ($h(n)$).

$$f(n) = g(n) + h(n)$$

2. Admissible heuristic: $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to reach the goal from n . (Optimistic)

3. We only perform a goal test when about to expand the node, not put the node in the frontier.

4. Euclidean and Manhattan distance are both admissible and consistent.

5. Optimality:

- Tree search version of A^* search is optimal when heuristic is admissible.
- Graph search version of A^* search is optimal when heuristic is consistent.

Completeness: A^* search is always complete.

Summary

Feature	Uninformed search	Informed search
Search Algorithms	BFS, DFS, UCS (Dijkstra's algorithm)	Greedy best-first search, A* search
Additional knowledge available?	No (explores blindly, only uses problem definition, i.e., start, actions, goal)	Yes (uses domain-specific knowledge/heuristics to guide search)
Complete?	BFS & UCS (Dijkstra's algorithm): Yes DFS: No (can get stuck in infinite paths)	Greedy best-first: No (can get stuck in local optima) A*: Generally yes (finite search space and non-negative edge cost)
Optimal?	BFS: Yes (if all step costs are equal/unweighted graph) DFS: No (finds any path, not necessarily shortest) UCS (Dijkstra's algorithm): Yes (finds optimal path in weighted graphs with non-negative costs)	Greedy best-first: No (may find suboptimal path instead) A*: Yes (with admissible & consistent heuristic)
Efficiency	Generally less efficient for large search spaces, as it explores broadly/deeply without guidance. May explore many irrelevant paths	Generally more efficient for large search spaces, as heuristics guide it towards the goal.
When to use	Small or simple problems No heuristic available	Larger or more complex problems When good heuristic is available

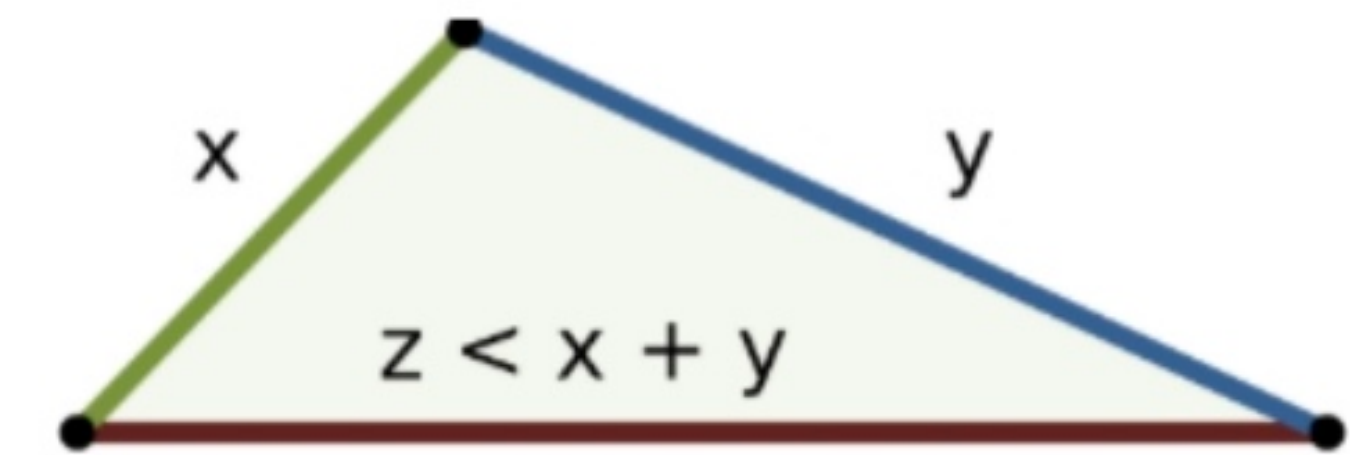
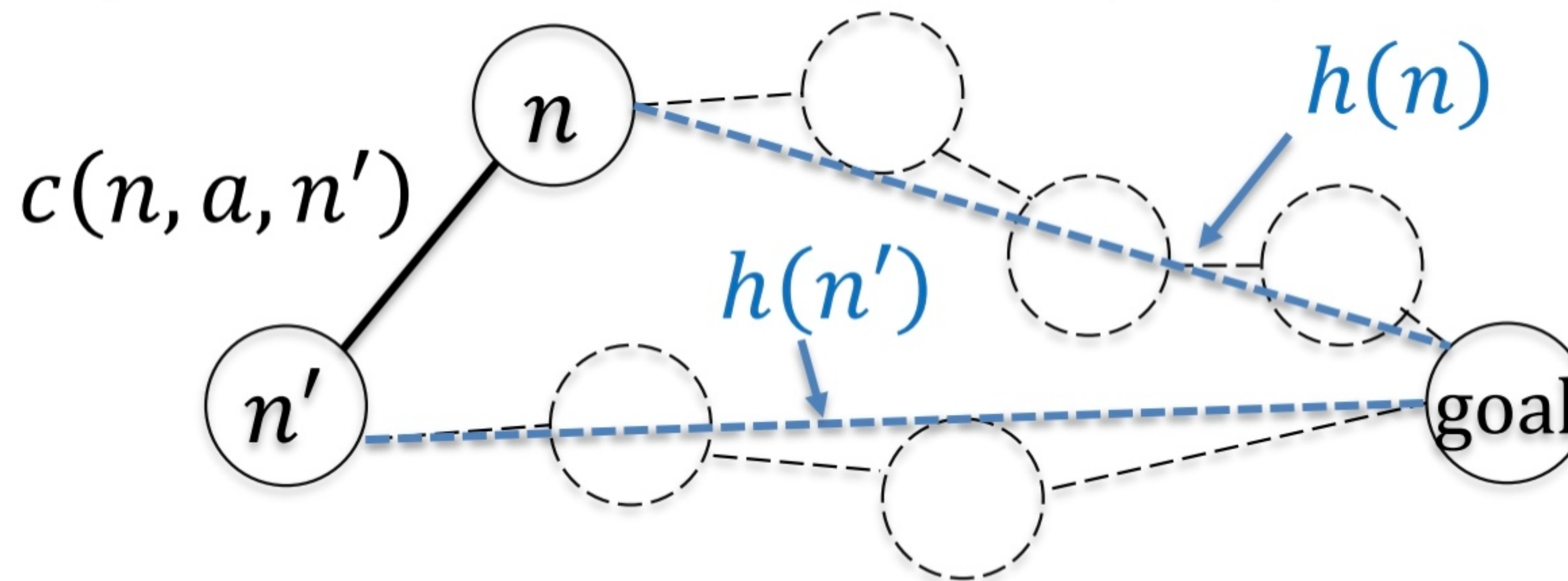
Consistent heuristics

A heuristic $h(n)$ is **consistent** (or monotonic) if for every node n , every successor n' of n generated by any action a :

$$h(n) \leq c(n, a, n') + h(n')$$

$c(n, a, n')$ is the actual cost of applying action a to go from state n to n'

Consistency is a version of triangle inequality.



Consistency is a **stricter requirement** than admissibility.